ElectroMap: EV Charging Station Finder

Project Report

Contents

1 Introduction **3**

2 Scenario Based Case Study **3**
2.1 User Registration and Authentication 3
2.2 Finding Nearby Charging Stations 3
2.3 Station Details and Decision Making 3
2.4 Saving Stations and Reviewing Experiences 3
2.5 Admin Management 4

3 System Requirements **4**
3.1 Software Requirements 4
3.2 Hardware Requirements 4

4 Project Architecture **5**
4.1 High Level Flow 5
4.2 Technical Architecture 5

5 ER Diagram Description **6**
5.1 Entities 6
5.2 Relationships 6
5.3 Primary and Foreign Keys 6

6 Key Features **6**

7 Roles and Responsibilities **7**
7.1 User 7
7.2 Admin 7

8 User Flow **8**
8.1 Customer Flow 8
8.2 Admin Flow 8

9 MVC Pattern **8**
9.1 Model Layer 8
9.2 Controller Layer 9
9.3 Routing Layer 9
9.4 Advantages of MVC 9

10 Project Setup and Configuration **9**
10.1 Folder Structure 9
10.2 Backend Setup 9
10.3 Backend Environment Variables 10
10.4 Frontend Setup 10
10.5 Frontend Environment Variables 10

11 Backend Development **11**
11.1 Backend Structure 11
11.2 API Routes 11
11.3 Security and Middleware 12

12 Database Development **12**
12.1 MongoDB Configuration 12
12.2 Schemas 13
12.3 Frontend Structure 13
12.4 Frontend Pages 14

13 Output Screens **17**

14 Testing and Validation **17**

15 Conclusion **18**

1 Introduction

ElectroMap is a full-stack web application designed to help electric vehicle users find reliable charging stations, check live charger availability, view station details, save preferred stations, and plan charging stops with better confidence. The application combines a React-based frontend, an Express.js backend, MongoDB data storage, JWT authentication, and real-time charger updates through Socket.IO.

The platform focuses on reducing range anxiety for EV drivers by giving them practical information before they reach a charging location. Users can browse stations on a map, compare charger types, check pricing, view amenities, read reviews, and open detailed pages for individual stations. Admin users can manage station records, charger data, users, reviews, and analytics from a protected dashboard.

2 Scenario Based Case Study

Consider a driver named Alex who uses an electric vehicle for daily travel and occasional long-distance trips. Before starting a trip, Alex needs to know where nearby chargers are available, whether those chargers are currently usable, and which locations are reliable based on reviews.

2.1 User Registration and Authentication

Alex creates an account in ElectroMap using a username, email address, and password. The backend stores the account securely, hashes the password, and issues a JWT token after login. The frontend stores the token and uses it when Alex accesses protected features such as the dashboard, saved stations, profile settings, and reviews.

2.2 Finding Nearby Charging Stations

After logging in, Alex opens the map page. The application requests location access and uses the current latitude and longitude to find nearby charging stations. Alex can also search or filter stations manually. The map interface helps Alex compare stations by distance, availability, charger type, rating, and location.

2.3 Station Details and Decision Making

Alex opens a station detail page to check the full address, charger types, number of available chargers, pricing, operating hours, amenities, images, and user reviews. This helps Alex decide whether the station is suitable for the trip.

2.4 Saving Stations and Reviewing Experiences

Alex saves reliable charging stations to the dashboard for quick access later. After using a station, Alex can submit a rating, comment, and tags such as Fast, Clean, Safe, or Reliable. These reviews improve the quality of information available to other EV users.

2.5 Admin Management

An admin manages the platform by adding and updating station records, checking charger status, monitoring users, and reviewing analytics. This keeps the charging network data useful and current.

3 System Requirements

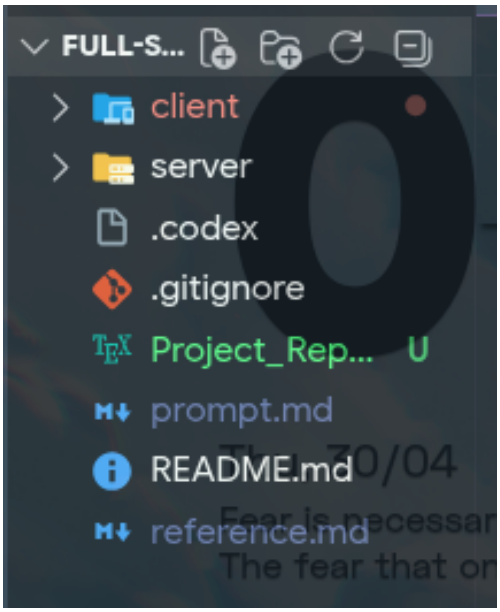
3.1 Software Requirements

- Operating System: Windows 10/11, macOS, or Linux.
- Node.js: Version 16 or above is required for the backend and frontend tooling.
- npm: Required for installing dependencies and running scripts.
- React.js: Used for building the frontend user interface.
- Vite: Used as the frontend development server and build tool.
- Express.js: Used for building the REST API.
- MongoDB: Used as the NoSQL database.
- Mongoose: Used for schema design and MongoDB interaction.
- Browser: Latest version of Chrome, Firefox, Edge, or another modern browser.
- Postman: Useful for API testing during development.
- Visual Studio Code: Recommended editor for development.
- MapTiler API Key: Required for live map rendering and location search features.

3.2 Hardware Requirements

- Processor: Intel Core i5 or AMD Ryzen 5 or better.
- RAM: Minimum 8 GB, with 16 GB recommended for smooth development.
- Storage: At least 1 GB free space for dependencies and project files.
- Display: 1366x768 or higher resolution.
- Internet: Required for package installation, map services, and external API access.

4 Project Architecture



ElectroMap follows a client-server architecture. The frontend is located in the `client` folder and the backend is located in the `server` folder.

4.1 High Level Flow

1. The user interacts with the React frontend.
2. The frontend sends HTTP requests to the Express API.
3. The backend validates requests, applies authentication or authorization where required, and calls controller logic.
4. Controllers communicate with MongoDB through Mongoose models.
5. The backend returns structured JSON responses to the frontend.
6. Socket.IO sends real-time charger status updates to subscribed clients.

4.2 Technical Architecture

Layer	Description
Frontend	React application built with Vite. It provides pages for home, map, station details, login, registration, user dashboard, and admin dashboard.
State Management	Zustand stores authentication, station, and map state on the client side.
Map Layer	MapLibre and MapTiler are used for map rendering, search, route display, and location-based features.
Backend API	Express.js application that exposes REST endpoints under `/api`.

Database	MongoDB stores users, stations, chargers, and reviews.
Authentication	JWT tokens protect private user and admin routes.
Real-Time Layer	Socket.IO supports live charger status updates.
Media Storage	Cloudinary configuration is available for uploaded station images and user avatars.

5 ER Diagram Description

The database design contains four main entities: User, Station, Charger, and Review.

5.1 Entities

- User: Stores account information such as username, email, password, avatar, role, saved stations, and account creation date.
- Station: Stores station name, address, city, state, country, coordinates, charger types, charger counts, pricing, operating hours, amenities, images, rating, verification status, and creator reference.
- Charger: Stores station reference, charger type, power output, connector type, status, and last updated timestamp.
- Review: Stores station reference, user reference, rating, comment, tags, and creation date.

5.2 Relationships

- One user can save many stations.
- One station can have many chargers.
- One station can have many reviews.
- One user can write many reviews.
- A review belongs to one user and one station.
- A station is added by one admin user.

5.3 Primary and Foreign Keys

MongoDB ObjectIds are used as primary identifiers. References between collections are represented using ObjectId fields such as `stationId`, `userId`, and `addedBy`.

6 Key Features

- User registration, login, logout, and profile update.
- JWT-protected user dashboard.
- Admin-only dashboard and management routes.

- EV station listing, search, filtering, and nearby station discovery.
- Interactive map page with station markers and route planning support.
- Station detail page with pricing, amenities, operating hours, images, chargers, and reviews.
- Save and unsave station functionality.
- Review creation, editing, and deletion.
- Charger status management with real-time update events.
- Admin analytics for stations, chargers, users, and reviews.
- Optional mock data mode for frontend demonstration.

7 Roles and Responsibilities

7.1 User

- Create and manage an account.
- Search for stations and view nearby charging locations.
- Check charger availability, station details, pricing, and amenities.
- Save favourite stations.
- Submit, edit, and delete personal reviews.
- Update profile information and avatar.

7.2 Admin

- Add, update, and delete charging stations.
- Manage station images and station metadata.
- View and manage users.
- View charger information and update charger status.
- View review data.
- Monitor platform analytics.

8 User Flow

8.1 Customer Flow

1. Register or log in.
2. Open the home page or map page.
3. Allow location access or search manually.
4. Browse charging stations.
5. Open a station detail page.
6. Save the station or navigate to it.
7. Submit a review after using the station.
8. Manage saved stations and reviews from the dashboard.

8.2 Admin Flow

1. Log in with an admin account.
2. Open the admin dashboard.
3. View analytics and system statistics.
4. Add or update stations.
5. Monitor chargers, users, and reviews.
6. Update user roles or station data when required.

9 MVC Pattern

The backend follows the Model-View-Controller pattern. In this REST API project, the routing layer acts as the view layer because it receives HTTP requests and maps them to controller functions.

9.1 Model Layer

The model layer is implemented with Mongoose schemas in the `server/models` folder. The main models are:

- `User.js`
- `Station.js`
- `Charger.js`
- `Review.js`

9.2 Controller Layer

Controllers are stored in `server/controllers`. They contain business logic for authentication, users, stations, chargers, reviews, and admin operations.

9.3 Routing Layer

Routes are stored in `server/routes`. They define API endpoints and connect each endpoint to the correct controller. Middleware is used for validation, authentication, authorization, rate limiting, and error handling.

9.4 Advantages of MVC

- Clear separation of concerns.
- Easier maintenance and debugging.
- Controllers and models can be tested independently.
- New features can be added by creating new routes, controllers, and models.
- Multiple developers can work on different layers with fewer conflicts.

10 Project Setup and Configuration

10.1 Folder Structure

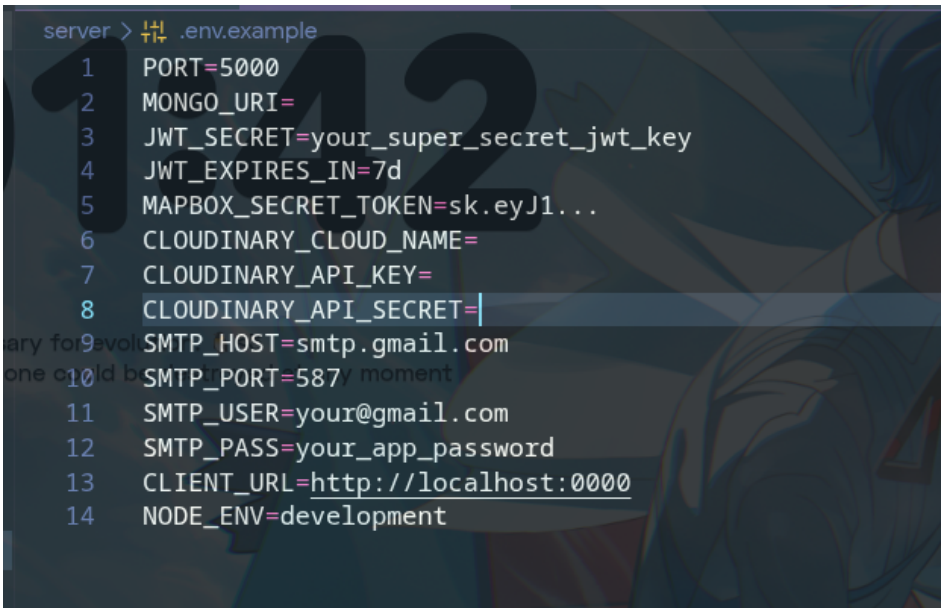
```
full-stack/
  client/
    src/
    public/
    package.json
  server/
    config/
    controllers/
    middleware/
    models/
    routes/
    services/
    utils/
    package.json
    server.js
```

10.2 Backend Setup

1. Open the server folder.
2. Install dependencies using `npm install`.
3. Create `server/.env`.
4. Add MongoDB and JWT configuration.

5. Run the backend using `npm run dev`.

10.3 Backend Environment Variables




```
PORT=5000
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_jwt_secret
JWT_EXPIRES_IN=7d
CLIENT_URL=http://localhost:5173
CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret
IOT_SECRET_TOKEN=optional_iot_token
```

10.4 Frontend Setup

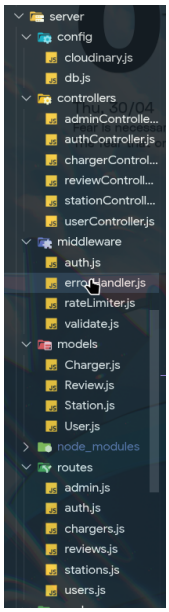
1. Open the client folder.
2. Install dependencies using `npm install`.
3. Create `client/.env` if custom environment values are required.
4. Run the frontend using `npm run dev`.

10.5 Frontend Environment Variables

```
VITE_API_BASE_URL=http://localhost:5000/api
VITE_SOCKET_URL=http://localhost:5000
VITE_MAPTILER_KEY=your_maptiler_key
VITE_USE_MOCK_DATA=false
```

11 Backend Development

11.1 Backend Structure



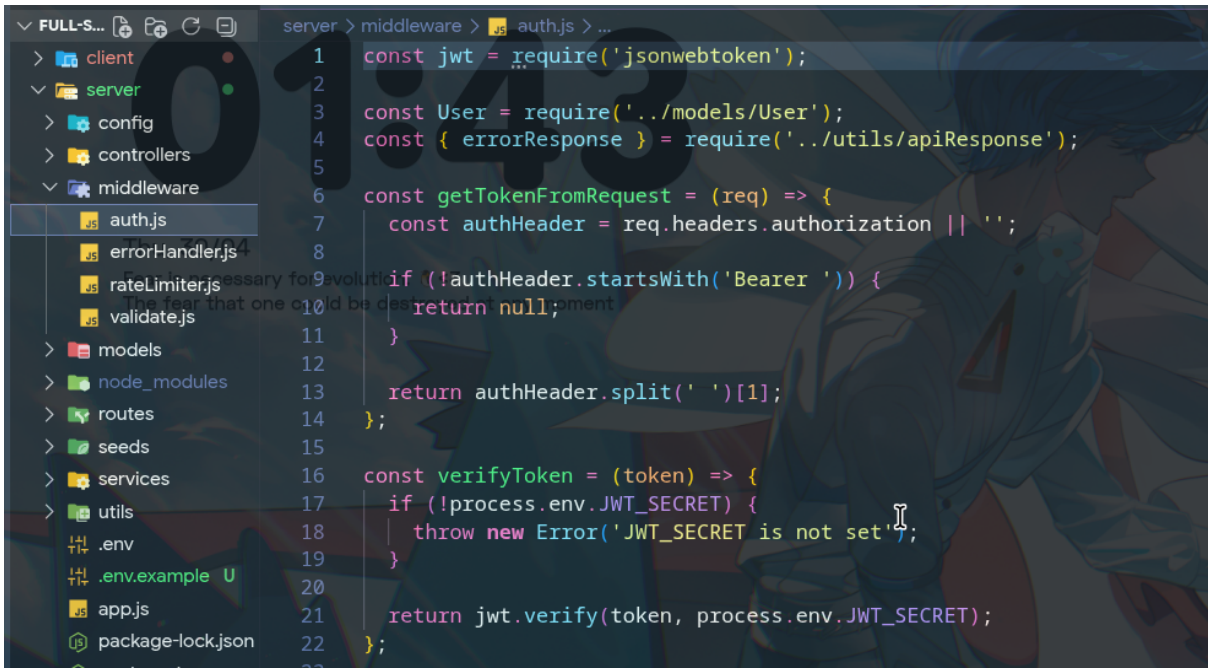
Path	Purpose
`server.js`	Starts the HTTP server, loads environment variables, connects to MongoDB, and initializes Socket.IO.
`app.js`	Configures Express middleware, CORS, routes, health check, and error handling.
`config/db.js`	Connects the backend to MongoDB.
`config/cloudinary.js`	Configures Cloudinary for image uploads.
`models/`	Contains Mongoose schemas.
`controllers/`	Contains business logic.
`routes/`	Contains API route definitions.
`middleware/`	Contains authentication, validation, rate limiting, and error handling middleware.
`services/socketService.js`	Handles Socket.IO subscriptions and update events.
`utils/`	Contains reusable response, validation, and geolocation helpers.

11.2 API Routes

Route Group	Main Endpoints
`/api/health`	Checks backend health.
`/api/auth`	Register, login, logout, current user, and profile update.
`/api/stations`	List stations, nearby stations, search, station detail, create, update, delete, save, and unsave.
`/api/chargers`	Get chargers by station and update charger status.

`/api/reviews`	Get station reviews, create review, update review, and delete review.
`/api/users`	Saved stations, user profile, and current user's reviews.
`/api/admin`	Admin stats, users, user roles, reviews, and chargers.

11.3 Security and Middleware



- Helmet adds common security headers.
- CORS allows configured frontend origins.
- Express rate limiting protects general and authentication routes.
- JWT middleware protects private routes.
- Admin middleware restricts admin-only endpoints.
- Zod validation utilities are available for request validation.
- Centralized error handling returns consistent error responses.

12 Database Development

12.1 MongoDB Configuration

The backend uses Mongoose to connect to MongoDB. The connection string is provided through the `MONGO_URI` environment variable. If the variable is missing, the database connection code reports an error.

12.2 Schemas

User Schema

The User schema stores username, email, hashed password, avatar URL, role, saved station references, and creation date. Passwords are hashed before saving, and a method is provided to compare login passwords.

Station Schema

The Station schema stores station identity, address, geospatial coordinates, charger types, charger availability counts, pricing, operating hours, amenities, images, ratings, verification status, and admin creator reference. It also defines geospatial, city, rating, and text search indexes.

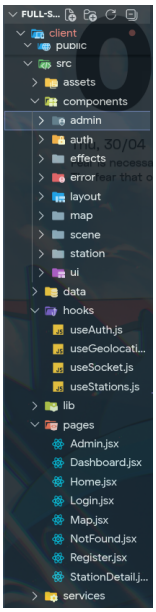
Charger Schema

The Charger schema stores the station reference, charger type, power output, connector type, current status, and last updated timestamp. Supported statuses are available, occupied, offline, and maintenance.

Review Schema

The Review schema stores the station reference, user reference, rating, comment, tags, and creation date. It recalculates station average rating and total review count when reviews are saved or deleted.

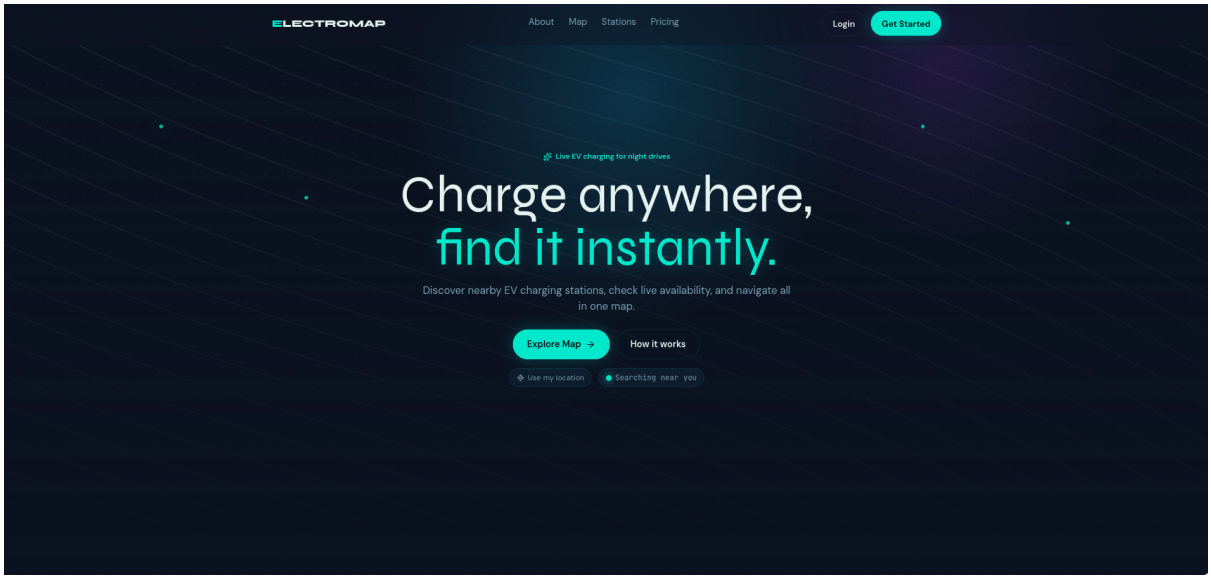
12.3 Frontend Structure



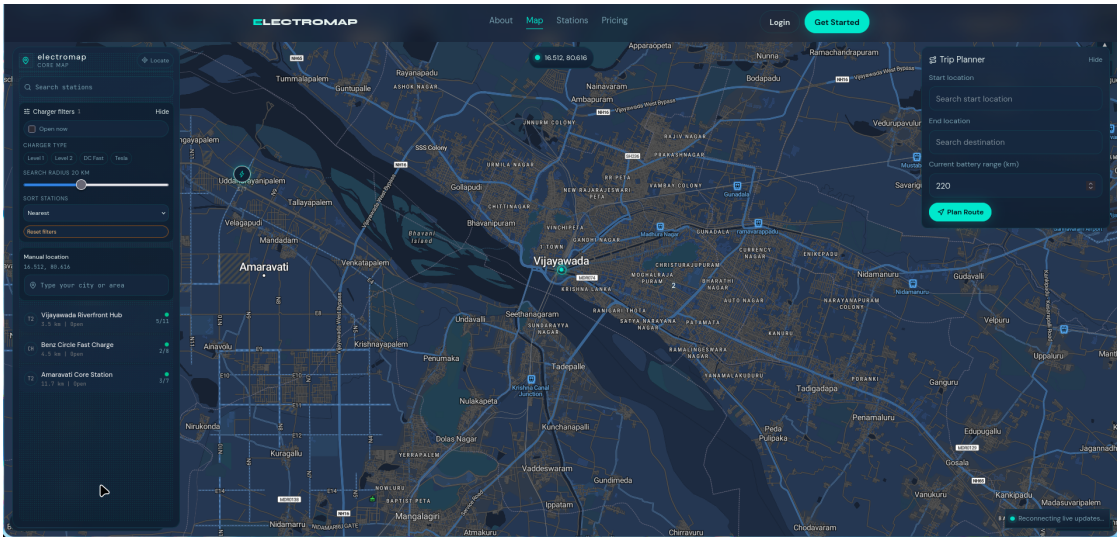
Path	Purpose
`src/App.jsx`	Defines application routes.
`src/pages/Home.jsx`	Landing page with station highlights and entry points.

`src/pages/Map.jsx`	Main map interface for station discovery and route planning.
`src/pages/StationDetail.jsx`	Detailed station information, chargers, reviews, and save actions.
`src/pages/Dashboard.jsx`	User dashboard for saved stations, reviews, and profile settings.
`src/pages/Admin.jsx`	Admin dashboard for platform management and analytics.
`src/pages/Login.jsx`	User login page.
`src/pages/Register.jsx`	User registration page.
`src/components/map/`	Map sidebar, map view, station markers, route planner, and route layer.
`src/components/station/`	Station cards and station drawer components.
`src/components/admin/`	Station form modal used by admins.
`src/components/ui/`	Reusable UI components.
`src/store/`	Zustand state stores.
`src/services/api.js`	Axios API client with auth token handling.

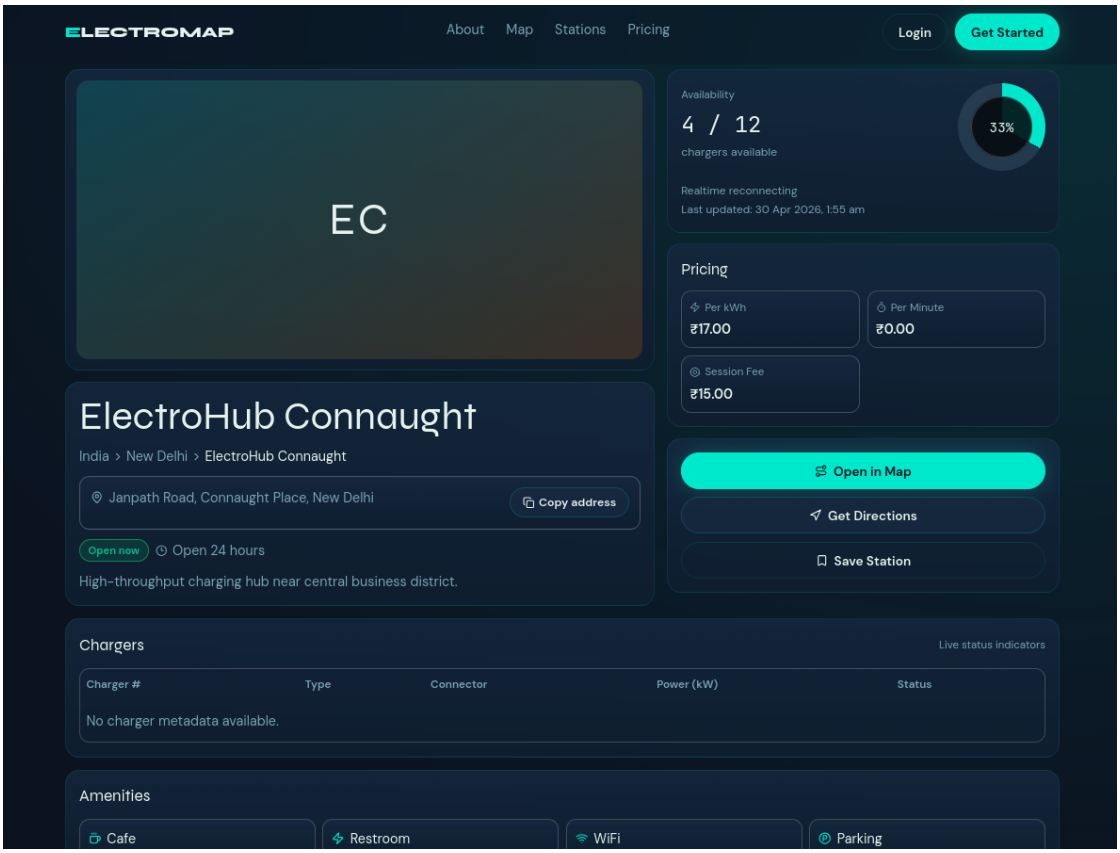
12.4 Frontend Pages



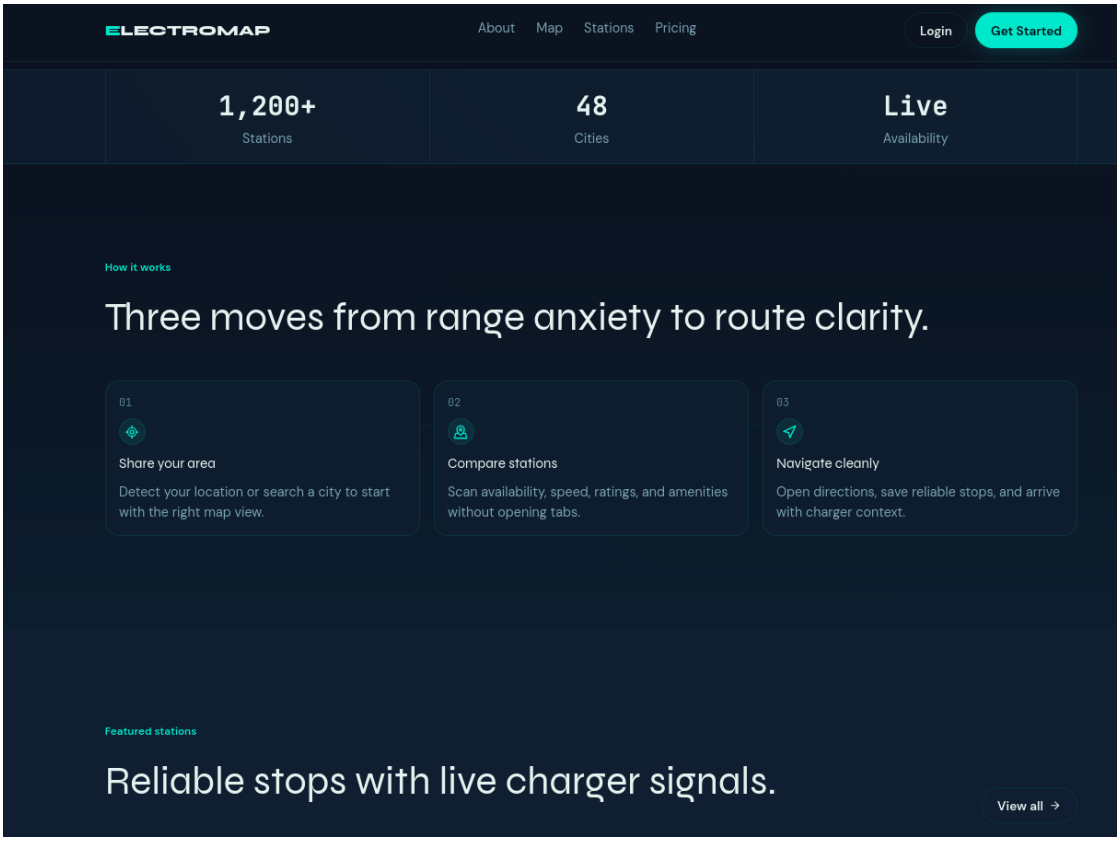
- Home Page: Presents the application purpose and featured charging stations.
- Map Page: Shows EV stations on a map with filtering, search, route planning, and station selection.



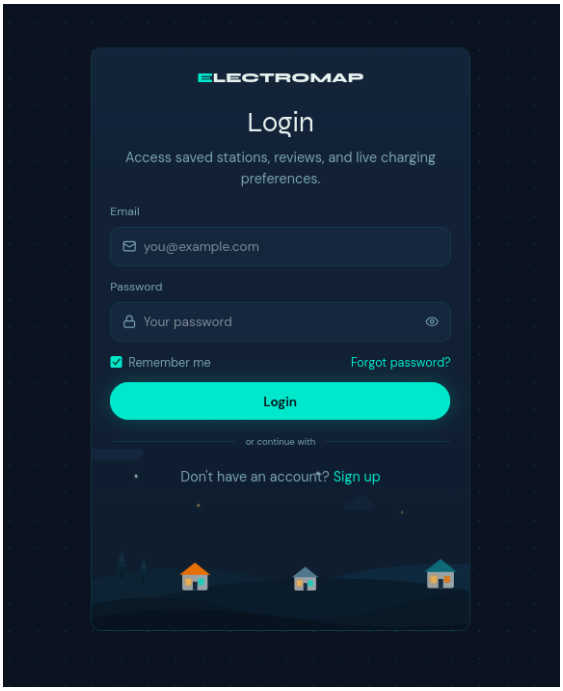
- Station Detail Page: Displays full station data, charger information, pricing, amenities, and reviews.



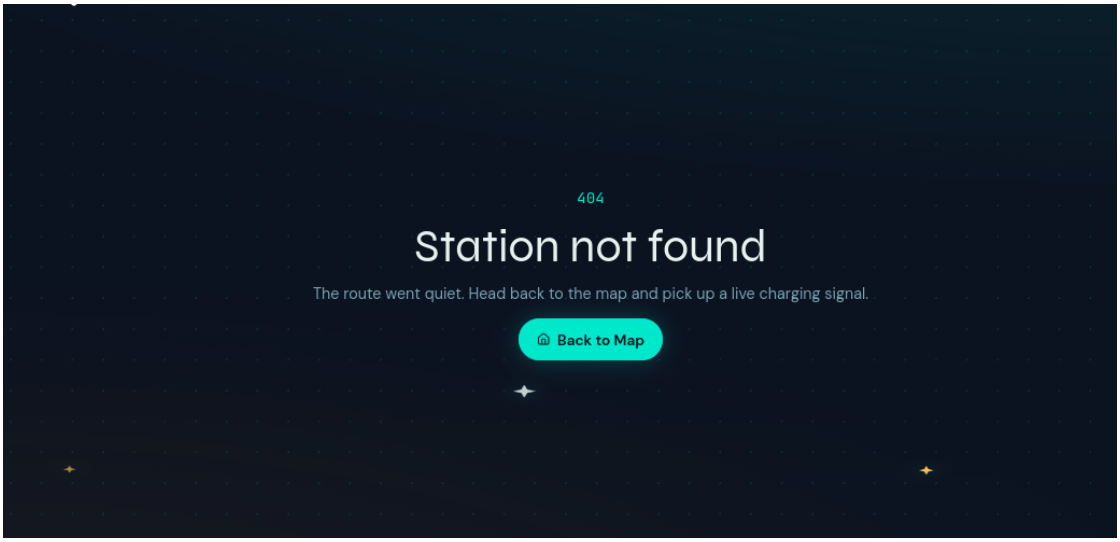
- Dashboard Page: Lets users manage saved stations, reviews, and profile settings.



- Admin Page: Lets admins manage stations, users, chargers, reviews, and analytics.
- Login and Register Pages: Handle user authentication flows.



- Not Found Page: Handles invalid routes.



13 Output Screens

The current application contains the following main screens:

- Home page
- Registration page
- Login page
- EV station map page
- Station detail page
- User dashboard
- Admin dashboard
- Admin station form modal
- Not found page

14 Testing and Validation

The frontend includes a lint script through ESLint. The backend currently has a placeholder test script. During development, APIs can be tested manually with Postman or another API client. Important test areas include:

- User registration and login.
- Protected user and admin routes.
- Station search and nearby station queries.
- Charger status updates.
- Review creation, update, deletion, and rating recalculation.

- Map rendering with a valid MapTiler key.
- Frontend behavior in mock data mode and live API mode.

15 Conclusion

ElectroMap provides a complete full-stack solution for EV charging station discovery and management. It supports user authentication, station browsing, interactive maps, saved stations, reviews, admin controls, MongoDB persistence, and real-time charger status updates. The project is structured in a scalable way, with separate frontend and backend folders, clear MVC-style backend organization, reusable frontend components, and environment-based configuration.